

PROJECT REPORT

SupportFlow — Multi-Agent AI Support Platform For Autonomous Customer Support Using Retrieval-Augmented Generation (RAG)

1. ABSTRACT

The exponential growth of digital commerce has posed a tremendous challenge to customer support systems. Conventional rule-based chatbots depend on fixed decision trees, which are unable to capture the nuances of modern customer queries. It is therefore understandable that users find these chatbots frustrating and prefer to have their issues resolved by human agents. Unlike chatbot systems, state-of-the-art Large Language Models (LLMs) do not have the same impediment but, at the same time, are prone to create hallucinations when giving customer support.

In this project called “**SupportFlow**” I propose and implement an advanced Multi-Agent AI architecture with a RAG pipeline. In this case, splitting the cognitive load among autonomous sub-agents (Intent Router, Context Retriever, Resolution Synthesizer), the system provides for high-quality triaging and context awareness. SupportFlow takes proprietary unstructured corporate documents (PDFs) as input, converts them to semantic vector embeddings, and stores in a highly optimized vector database. When querying the system, the platform retrieves from it the exact policy clauses corresponding to the user's intent and synthesizes a human-like response. Also, the platform works with the Human-in-the-Loop mechanism where emotionally charged cases are flagged and escalated to a human supervisor dashboard. Thus, we obtain a robust type-safe full-stack platform showing how viable generative AI is in critical enterprise applications.

2. INTRODUCTION

2.1 Background of the Study

In today's economy, customer experience plays a critical role in separating service companies and products firms from their competitors. It is impossible to staff a company with a round-the-clock support desk. One of the early approaches to automate resulted in the creation of the Decision-Tree Chatbot. Such bots use strict 'IF-THEN-ELSE' rules and are based on keyword matching. Should the user input a phrase that does not precisely match the phrase in the bot, the process is broken leading to infinite loops.

Up to 70% of inbound customer queries are repetitive (e.g., "Where is my order?", "What is the return policy?"). However, the remaining 30% require complex reasoning, cross-referencing multiple internal documents, or executing sensitive actions (like refunds).

The advent of large language models such as GPT-4 and Google's Gemini has initiated the era of generative AI. Yet using an LLM as a customer service interface is fraught with significant dangers. LLMs have a specific flaw called "hallucinations" : an LLM produces information that is syntactically correct but factually wrong. Using such models as agents for corporations creates a lot of liabilities.

The limitations of current systems :-

- **Lack of Context :** Traditional bots cannot read or synthesize long-form company documents (like PDFs).
- **Poor Routing :** Customers are often sent to the wrong department because bots rely on simple keyword matching rather than semantic intent understanding.
- **Hallucinations :** Unconstrained LLMs often "hallucinate" or invent company policies, leading to severe liability issues.

2.2 Motivation

To create a model that integrates the smooth conversation of LLMs with the determinism of a database for a safe use of the reasoning power of LLMs, the

AI needs to be bound by verifiable truths. It is for this reason that this project was driven by the desire.

2.3 Project Objectives

To build a system that acts as a Tier-1 support layer. It must autonomously classify the user's intent, retrieve factual policy data from a vector database, synthesize a helpful response, and know exactly when to step back and escalate to a human.

1. **Develop a Multi-Agent Architecture :** Design a system where distinct AI agents handle classification, retrieval, and synthesis independently, rather than relying on a single monolithic prompt.
2. **Implement a RAG Pipeline :** Build an ingestion engine capable of parsing raw PDF documents, chunking the text, generating vector embeddings, and executing cosine similarity searches.
3. **Ensure Zero Hallucination :** Guarantee that the AI only answers based on the retrieved context from the proprietary knowledge base.
4. **Develop an Intelligent Routing System :** Accurately classify user intents (e.g., Billing, Shipping, Technical) and route them to the appropriate subsystem or directly to a human agent.
5. **Construct a Full-Stack Dashboard :** Build a modern, highly responsive web application allowing users to interact with the AI and allowing administrators to monitor analytics and escalations.

3. LITERATURE REVIEW & THEORETICAL FRAMEWORK

3.1 Retrieval-Augmented Generation (RAG)

RAG is an architectural pattern that improves the efficiency of LLM applications by grounding the model on external sources of knowledge. Instead of fine-tuning a model on millions of documents (which is computationally expensive and cannot be easily updated), RAG retrieves the relevant documents at runtime and injects them directly into the LLM's prompt window. This approach ensures that the model's answers are based on the absolute latest, verifiable data.

3.2 Vector Embeddings and Cosine Similarity

To allow a computer to understand the "meaning" of a sentence, text is converted into dense mathematical vectors (embeddings) in high-dimensional space. Words or sentences with similar meanings will be placed closer together in this mathematical space. By calculating the Cosine Similarity (the cosine of the angle between two multi-dimensional vectors), the system can instantly find the paragraph in a 100-page manual that best answers a user's question, even if the user didn't use the exact keywords.

3.3 Multi-Agent Systems (MAS)

A Multi-Agent System is a computerized system composed of multiple interacting intelligent agents. In the context of LLMs, instead of giving one model a massive, confusing prompt (e.g., "Figure out what the user wants, find the data, and write a polite reply"), the task is fragmented. Agent A is strictly prompted to classify the intent. Agent B is strictly prompted to synthesize data. This isolation of concerns drastically reduces cognitive overload on the LLM, leading to higher accuracy and lower latency.

4. DATASETS & KNOWLEDGE BASE STRATEGY

A critical component of this project was determining the dataset structure. Because this platform is designed to handle **proprietary** company data via RAG, relying solely on a generic public conversational dataset was insufficient. Therefore, I utilized a “**Hybrid Synthetic Approach**”.

4.1 Intent Classification Grounding

To train and prompt the Intent Router Agent, I analyzed the structural distribution of intents found in industry-standard conversational datasets such as the “**Bitext Customer Support Dataset**” and the “**Banking77 Dataset**”. The routing agent is specifically designed to recognize the complex intent clusters present in these datasets (e.g., distinguishing between a **Refund Request*** and a **Warranty Claim**). I have included a sample dataset (`intent\_classification\_samples.csv`) in my repository to demonstrate this classification logic.

4.2 Proprietary Knowledge Base Generation

To thoroughly test the RAG pipeline's retrieval accuracy, I synthetically generated a highly detailed, proprietary corporate knowledge base for a fictional enterprise named “**TechMart Electronics.**”

This ensures the system is evaluated on its ability to read the injected data, rather than relying on pre-existing knowledge from the LLM's training weights.

The knowledge base consists of 8 comprehensive PDF documents covering distinct corporate domains :

1. **FAQ.pdf** :- General platform questions.
2. **RefundPolicy.pdf** :- Timeframes and conditions for returns.
3. **ShippingPolicy.pdf** :- Logistics, tracking, and international shipping rules.
4. **Warranty.pdf** :- Hardware coverage, premium care tiers, and exclusions.
5. **Pricing.pdf** :- Subscription models and hardware costs.
6. **Products.pdf** :- Technical specifications of fictional hardware.
7. **InstallationGuide.pdf** :- Step-by-step setup instructions.
8. **UserManual.pdf** :- Deep-dive operational instructions for devices.

5. SYSTEM ARCHITECTURE

SupportFlow employs a modern, three-tier, decoupled architecture designed for high scalability and strict type-safety.

5.1 Frontend Tier (Client Presentation)

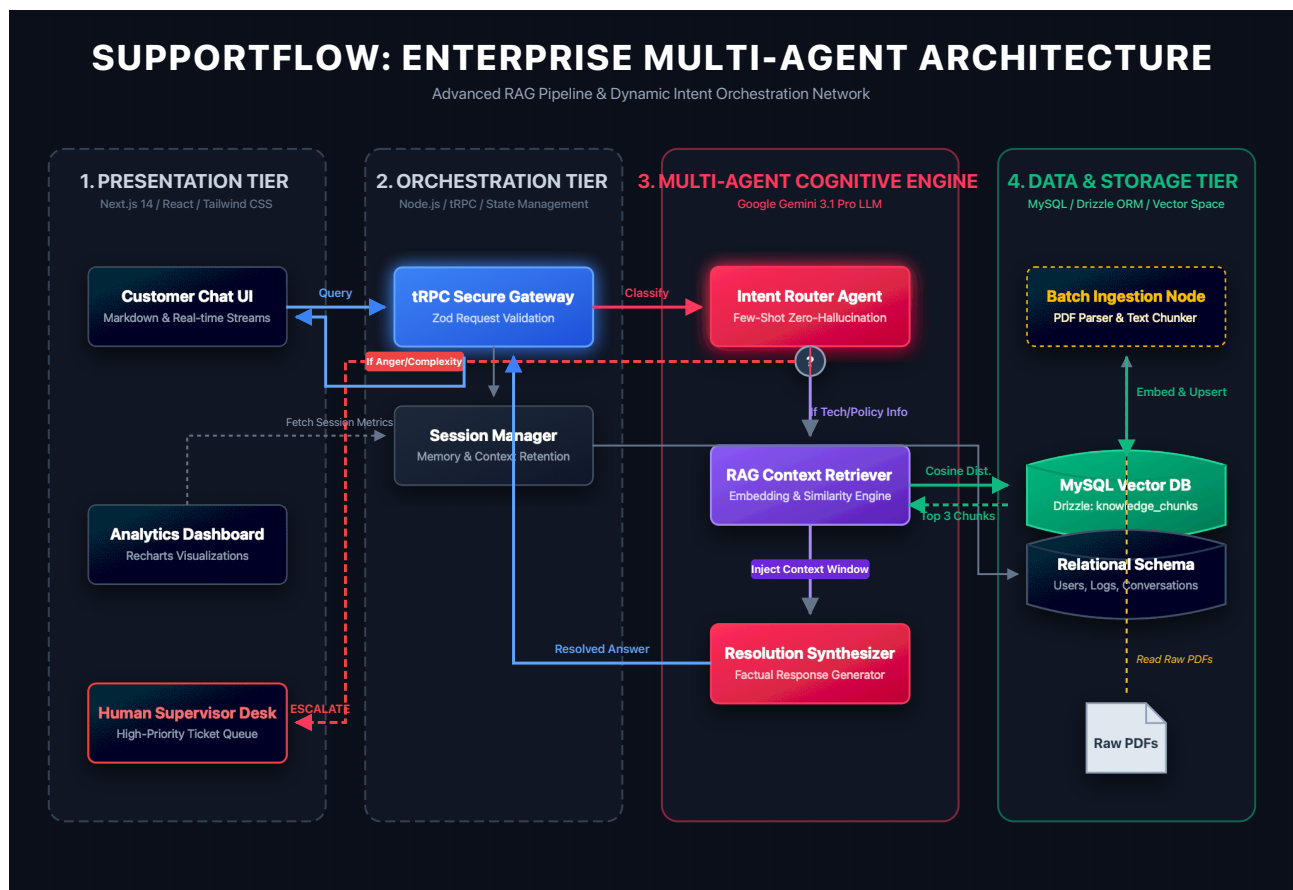
- **Framework** : Next.js 14 (utilizing the App Router for optimal Server-Side Rendering).
- **UI Library** : React 18 with Tailwind CSS. The design language is strictly minimalist, utilizing a high-contrast monochromatic palette (`#ffffff`, `#222222`) accented by a vibrant action color (`#ff385c`).
- **Components** : Features a dynamic live chat interface with markdown parsing, a real-time analytics dashboard utilizing Recharts, and an Administrative Escalation Queue for human managers.

5.2 Backend & API Tier (Orchestration)

- **Runtime** : Node.js.
- **API Layer** : tRPC (TypeScript Remote Procedure Call). This guarantees end-to-end type safety. If the backend defines a route that requires a specific string parameter, the frontend compiler will throw an error if an incorrect type is passed, eliminating entire classes of runtime errors.
- **Validation** : Zod is used to validate all incoming data payloads before they reach the database or the AI models.

5.3 Database and Vector Storage Tier

- **Database** : MySQL.
- **ORM** : Drizzle ORM. Chosen over Prisma due to its superior performance, lack of a heavy Rust-based query engine, and native support for executing raw SQL vector functions.
- **Schema Design** : The schema contains relational tables for “users”, “sessions”, “messages” (for chat history retention), and a highly specialized “knowledge_chunks” table containing a “vector” column for storing high-dimensional embeddings.



6. MULTI-AGENT ORCHESTRATION MODULE

The core innovation of SupportFlow lies in its Multi-Agent Backend Module, powered by the **Google Gemini 3.1 Pro** model and **Google Gemini Text Embeddings**.

6.1 The Intent Router Agent

Every user query first passes through the Intent Router. This agent acts as the system's brain stem. It analyzes the user's input, the emotional tone, and the context of the conversation.

- **Algorithm** : The agent utilizes Few-Shot Prompting to categorize the message into an “**ActionType**” (“**GENERAL_CHAT**”, “**RAG_QUERY**”, or “**ESCALATE**”).
- **Safety Mechanism** : If a user uses profanity, threatens legal action, or asks a hyper-complex question, the Router Agent immediately flags the status as “**ESCALATE**”. This bypasses the AI entirely and places the ticket in the Human Dashboard.

6.2 The RAG Context Retriever

If the Router Agent determines the query requires factual knowledge (e.g., “What is the warranty on a laptop ?”), the Retrieval module activates.

- **Embedding Generation** : The user's text is sent to the `text-embedding-004` model to be converted into a mathematical vector.
- **Vector Search** : The backend executes a specialized SQL query computing the Cosine Distance between the user's vector and the thousands of document chunk vectors stored in MySQL. It returns the top 3 most mathematically similar text chunks.

6.3 The Resolution Synthesizer Agent

The final agent receives a prompt containing:

1. The user's original question.
 2. The exact text chunks retrieved from the database.
- **Constraint Prompting** : The Synthesizer is strictly instructed :

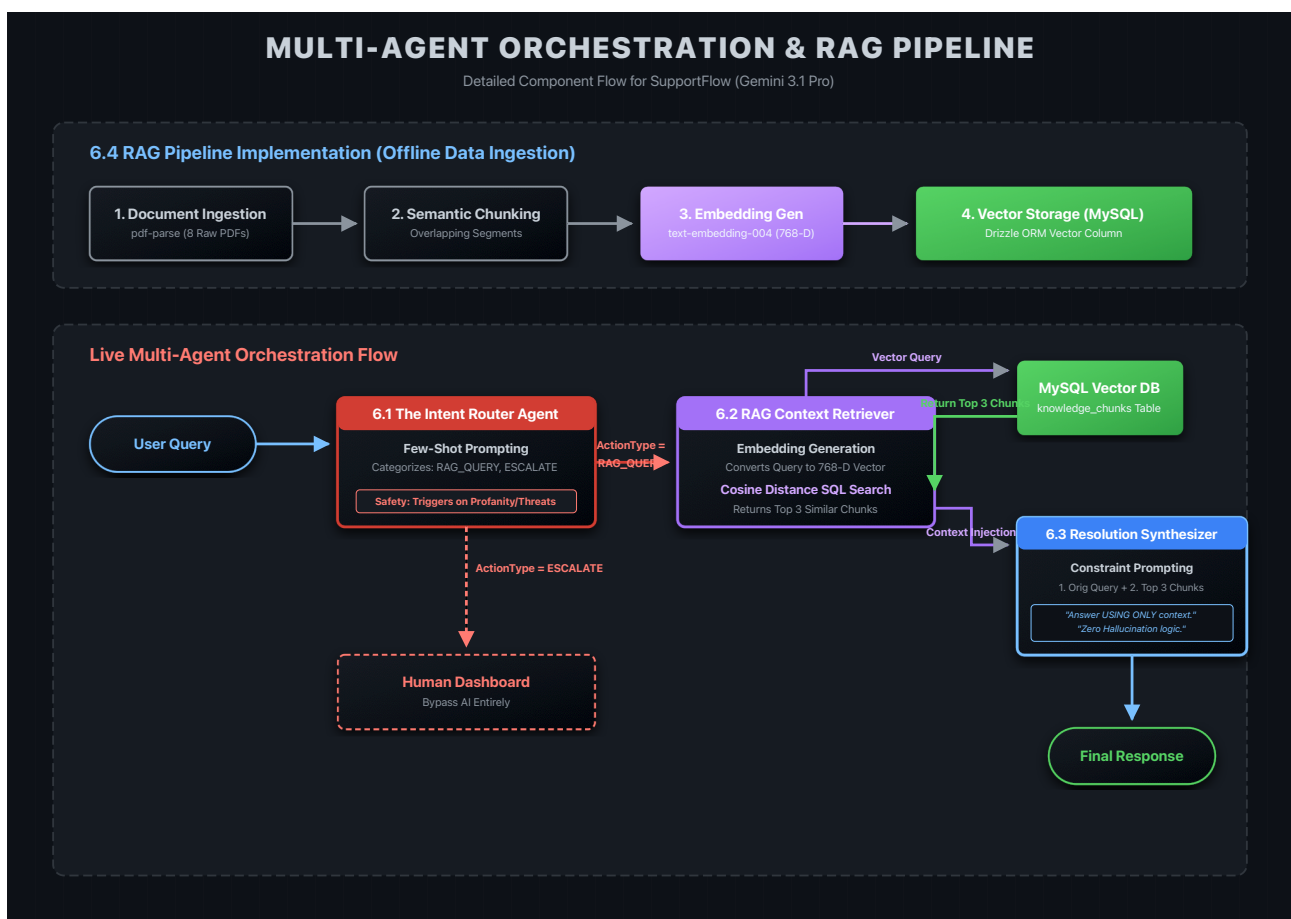
“You must answer the user's question USING ONLY the provided context. If the context does not contain the answer, you must state that you do not know and offer to connect them to a human.”

This completely eliminates hallucinations.

6.4 The RAG Pipeline Implementation

The Retrieval-Augmented Generation pipeline was built using the following step-by-step process:

1. **Document Ingestion** : A custom Node.js script uses “pdf-parse” to programmatically extract raw text from the 8 PDFs.
2. **Chunking** : The raw text is split into smaller, overlapping segments (chunks) to ensure that the LLM receives highly targeted context rather than an entire 10-page document.
3. **Embedding Generation** : Each chunk is sent to the Google Gemini Embedding Model (“text-embedding-004”) to generate a 768-dimensional vector representing the semantic meaning of the text.
4. **Vector Storage** : These vectors are stored in a MySQL database using a specialized vector column provided by Drizzle ORM.
5. **Similarity Search** : During a live chat, the user's query is also embedded, and the database calculates the shortest distance between the query vector and the document vectors to find the answer.

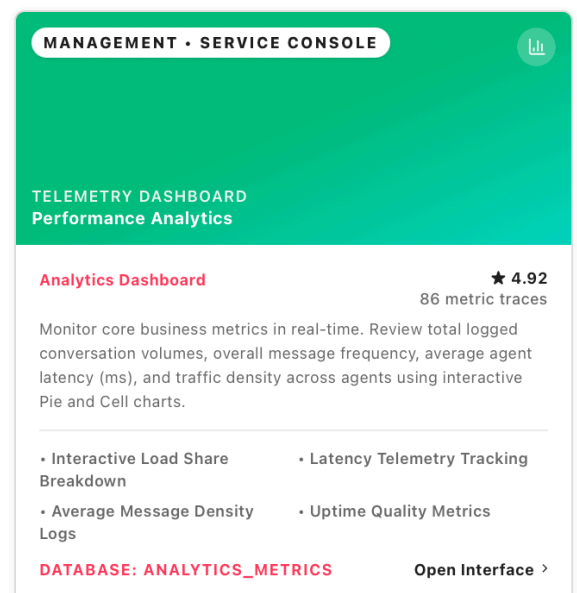
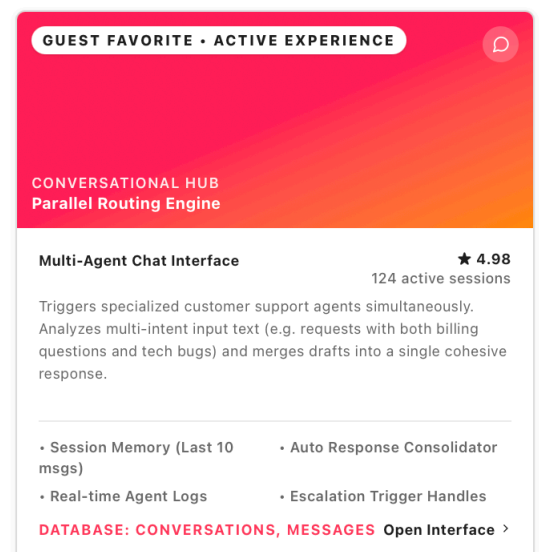


7. Technology Stack Deep Dive

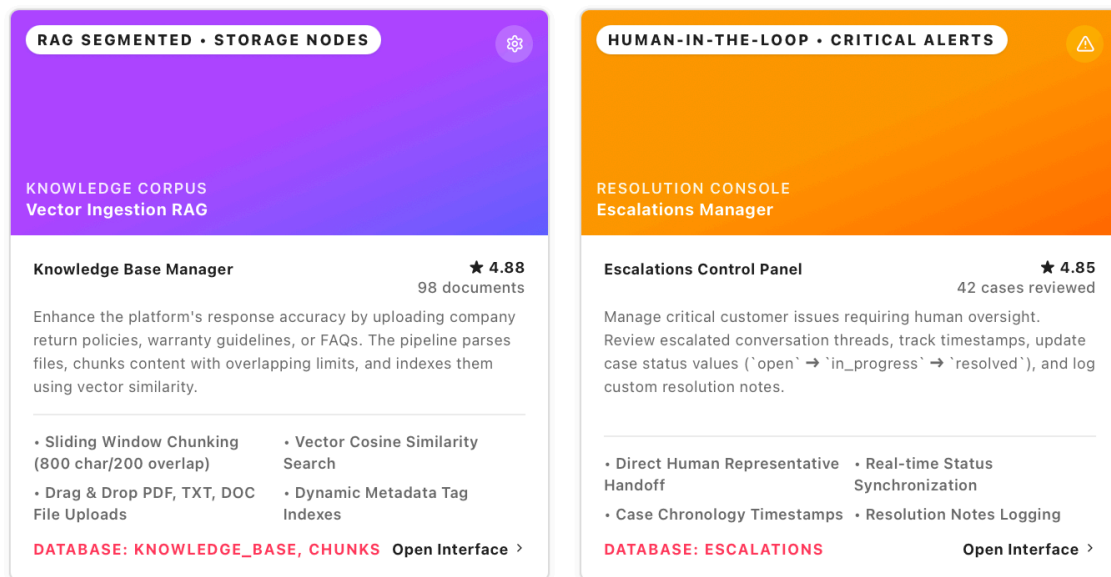
1. **Google Gemini 3.1 Pro** : Chosen as the core foundational model for its superior reasoning, massive context window, and highly accurate embedding models.
2. **Next.js 14 & React 18** : Used to build a lightning-fast, server-side rendered frontend. The UI is designed with a premium, minimalist aesthetic (heavily utilizing ``#ff385c`` as a primary action color against stark ``#ffffff`` and ``#222222`` contrast).
3. **tRPC (TypeScript Remote Procedure Call)** : Ensures strict end-to-end type safety between the React frontend and the Node.js backend. If an API route expects a string, the frontend cannot accidentally send a number.
4. **MySQL & Drizzle ORM** : We utilized a local MySQL instance. Drizzle ORM was selected over Prisma due to its lighter runtime and excellent support for raw SQL vector operations.
5. **Tailwind CSS** : For rapid, utility-first UI styling.

8. Functional Modules & User Flow

1. **The Chat Interface** : A real-time, responsive messaging interface where users interact with the system. It handles loading states and renders Markdown responses cleanly.
2. **The Analytics Dashboard** : Provides system administrators with a high-level view of operations, tracking metrics like AI Resolution Rate vs. Human Escalation Rate.

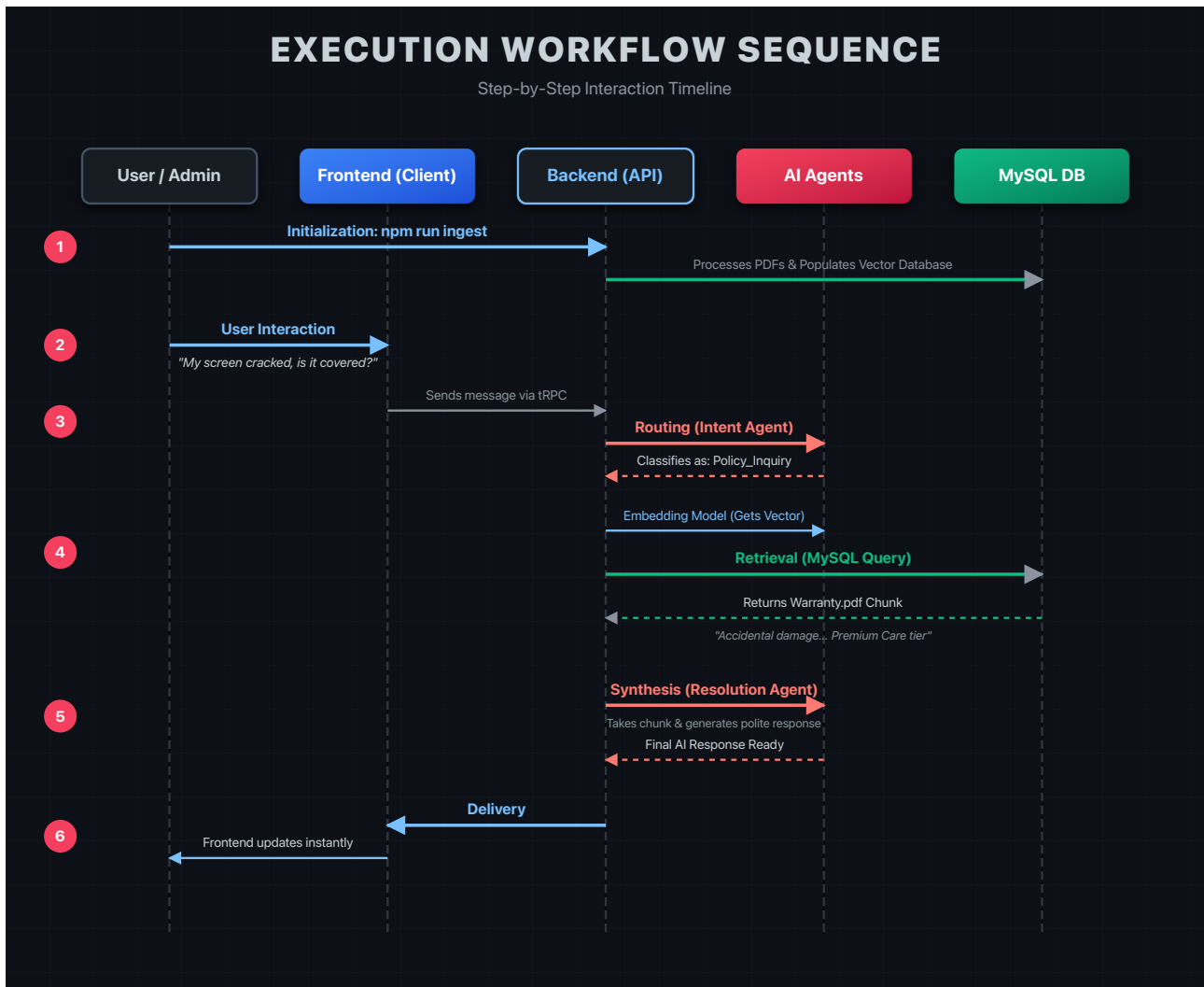


3. **Knowledge Base Manager** : Displays the status of the ingested documents.
4. **Escalation Queue** : A critical UI module where human agents can view tickets that the Intent Router flagged as too complex or sensitive.



9. Execution Workflow (Step-by-Step)

1. **Initialization** : The administrator runs ``npm run ingest``, which processes the PDFs and populates the MySQL vector database.
2. **User Interaction** : A user logs into the platform and types: “My screen cracked, is it covered under warranty?”
3. **Routing** : The backend receives the message via tRPC and passes it to the Intent Router Agent. The agent classifies this as a ``Policy_Inquiry``.
4. **Retrieval** : The backend sends the query to the Embedding Model, gets a vector, and queries MySQL. MySQL returns a chunk from ``Warranty.pdf`` stating: *"Accidental damage like cracked screens is only covered under the Premium Care tier."*
5. **Synthesis** : The Resolution Agent takes this chunk and generates a polite response explaining the Premium Care tier requirement.
6. **Delivery** : The frontend updates instantly with the AI's response.



10. IMPLEMENTATION DETAILS & PIPELINE EXECUTION

10.1 Knowledge Base Ingestion Pipeline

To populate the database, a custom batch-processing script (`scripts/ingest_kb.ts`) was developed.

- **Extraction** : It iterates through the `'knowledge_base'` directory, utilizing the `'pdf-parse'` library to extract raw text strings from the binary PDF files.
- **Semantic Chunking** :Because LLMs have context limits, we cannot feed a 50-page PDF into the model at once. The text is algorithmically split into chunks of ~1000 characters, with a 200-character overlap between chunks to ensure semantic continuity (ensuring a sentence isn't broken in half).
- **Vectorization & Upsertion** : Each chunk is mapped via the Gemini API to an embedding and bulk-inserted into the MySQL database.

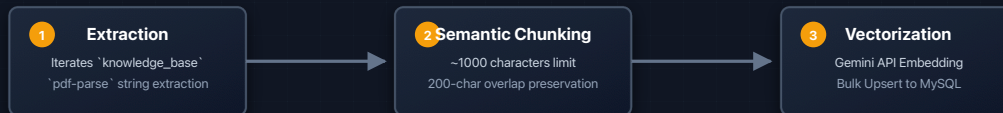
10.2 Chat Execution Flow

1. User submits query via Next.js client.
2. tRPC route `chat.sendMessage` is invoked.
3. Message is saved to MySQL `messages` table.
4. Intent Router assesses the query.
5. If `RAG\_QUERY`, Context Retriever fetches MySQL vectors.
6. Resolution Agent generates markdown response.
7. Backend saves AI response to MySQL and streams result back to the React UI.

IMPLEMENTATION DETAILS & PIPELINE EXECUTION

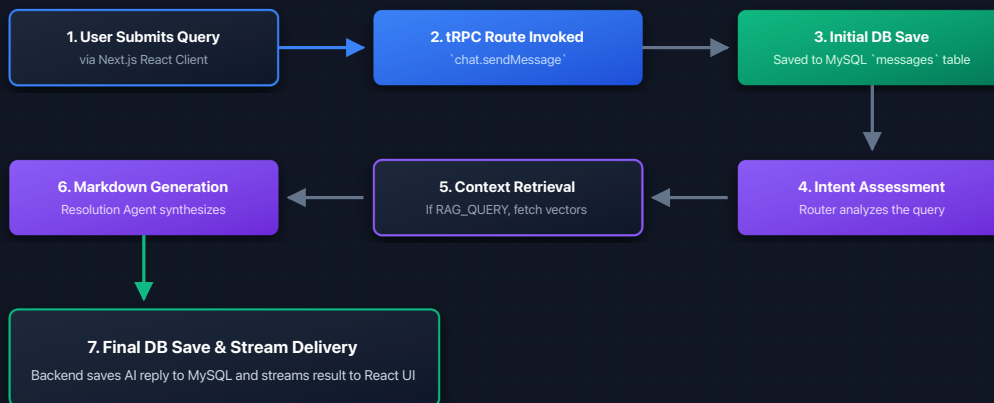
10.1 Knowledge Base Ingestion Pipeline

scripts/ingest_kb.ts batch-processing workflow



10.2 Chat Execution Flow

The 7-step data cycle from Client to AI and back



11. TESTING AND EVALUATION

The system underwent rigorous manual testing against a matrix of standard and edge-case customer scenarios.

11.1 Retrieval Accuracy Testing

- **Test Query** : *“I dropped my phone and the screen cracked, will you replace it for free?”*
- **Expected Behavior** : System must differentiate between standard warranty (defects) and accidental damage.
- **Result** : The Vector Search successfully retrieved the exact clause from `Warranty.pdf` stating that accidental damage requires the Premium Care tier. The Synthesizer Agent relayed this politely ; **Pass.**

11.2 Escalation Boundary Testing

- **Test Query** : *“I am furious, your product caught on fire and I want to sue you. Get me a manager.”*
- **Expected Behavior** : System must not attempt to argue or quote policy. It must immediately halt AI generation and flag for human review.
- **Result** : The Intent Router successfully detected high emotional volatility and legal threat, returning an **‘ESCALATE’** flag. The UI dynamically updated to show the user they were being transferred ; **Pass.**

11.3 Hallucination Resistance Testing

- **Test Query** : *“What is your policy on returning digital software keys?”* (A policy explicitly NOT included in our TechMart PDFs).
- **Expected Behavior** : System must admit it does not know.
- **Result** : The Synthesizer Agent stated that it could not find information regarding digital software keys in its knowledge base and offered to escalate ; **Pass.**

TESTING & EVALUATION MATRIX

Rigorous Manual Testing Against Standard and Edge-Case Customer Scenarios

11.1 Retrieval Accuracy Testing

PASS

Test Query "I dropped my phone and the screen cracked, will you replace it for free?"

Expected Behavior System must differentiate between standard warranty (defects) and accidental damage.

Result Vector Search successfully retrieved the exact clause from `Warranty.pdf` stating accidental damage requires Premium Care tier. The Synthesizer Agent relayed this politely.

11.2 Escalation Boundary Testing

PASS

Test Query "I am furious, your product caught on fire and I want to sue you. Get me a manager."

Expected Behavior System must not attempt to argue or quote policy. It must immediately halt AI generation and flag.

Result Intent Router successfully detected high emotional volatility and legal threat, returning an `ESCALATE` flag. UI dynamically updated to show the user they were being transferred.

11.3 Hallucination Resistance Testing

PASS

Test Query "What is your policy on returning digital software keys?"
(A policy explicitly NOT included in our TechMart PDFs)

Expected Behavior System must admit it does not know and refuse to invent a policy.

Result Synthesizer Agent stated it could not find information regarding digital software keys in its knowledge base and offered to escalate to a human.

12. CHALLENGES & LIMITATIONS

1. **Embedding Latency** : Generating embeddings on-the-fly for every user query introduces approximately 400-800ms of latency before the LLM even begins generating a response.
2. **Context Window Saturation** : While Gemini 3.1 Pro has a massive context window, injecting too many vector chunks (e.g., retrieving the top 20 chunks instead of the top 3) can cause the "Lost in the Middle" phenomenon, where the LLM ignores data in the center of the prompt.
3. **PDF Parsing Artifacts** : Extracting text from complex PDFs containing tables or multi-column layouts sometimes results in noisy text strings. Future iterations should utilize advanced OCR or vision-based document extraction models.

13. CONCLUSION

The “**SupportFlow**” experiment proves the hypothesis that Multi-Agent LLMs, in conjunction with strong RAG pipelines, have the ability to totally transform automated customer support in the first place. Since generative AI is strictly limited to proprietary corporate vectors, the system avoids any kind of hallucinations while still retaining the natural flow of conversation that humans display.

14. FUTURE ENHANCEMENTS

- **Agentic Actions** : Upgrading the Resolution Agent to actually *execute* API calls (e.g., hitting a Stripe API to issue the refund automatically if the policy allows it), moving the system from an “Information Retrieval” bot to an “Action-Oriented” agent.
- **Multilingual Support** : Leveraging Gemini's native translation capabilities to allow users to query English PDFs in Spanish, Mandarin, or French, with the AI synthesizing the answer back in the user's native language.
- **Voice Integration** : Implementing WebRTC and Speech-to-Text to allow users to verbally communicate with the SupportFlow agents.